5 Java concurrency fundamentals

This chapter covers

- Concurrency theory
- Block-structured concurrency
- Synchronization
- The Java Memory Model (JMM)
- Concurrency support in bytecode

Java has two, mostly separate concurrency APIs: the older API, which is usually called *block-structured concurrency* or *synchronization-based concurrency* or even "classic concurrency," and the newer API, which is normally referred to by its Java package name, `java.util.concurrent`.

In this book, we're going to talk about both approaches. In this chapter, we'll begin our journey by looking at the first of these two approaches. After that, in the next chapter, we'll introduce `java.util.concurrent`. Much later, we'll return to the subject of concurrency in chapter 16, "Advanced Concurrent Programming," which discusses advanced techniques, concurrency in non-Java JVM languages, and the interplay between concurrency and functional programming.

Let's get started and meet the classic approach to concurrency. This was the only API available until Java 5. As you might guess from the alternative name, "synchronization-based concurrency," this is the language-level API that is built into the platform and depends upon the `synchronized` and `volatile` keywords.

It is a low-level API and can be somewhat difficult to work with, but it is very much worth understanding. It provides a solid foundation for the chapters later in the book that explain other types and aspects of concurrency.

In fact, correctly reasoning about the other forms of concurrency is very difficult without at least a working knowledge of the low-level API and concepts that we will introduce in this chapter. As we encounter the relevant topics, we will also introduce enough theory

to illuminate the other views of concurrency that we'll discuss later in the book, including when we meet concurrency in non-Java languages.

To make sense of Java's approach to concurrent programming, we're going to start off by talking about a small amount of theory. After that, we'll discuss the impact that "design forces" have in the design and implementation of systems. We'll talk about the two most important of these forces, *safety* and *liveness*, and mention some of the others.

An important section (and the longest one in the chapter) is the detail of block-structured concurrency and an exploration of the low-level threading API. We'll conclude this chapter by discussing the Java Memory Model (JMM), and then using the bytecode techniques that we learned in chapter 4 to understand the real source of some common complexities in concurrent Java programming.

5.1 Concurrency theory primer

Let's get started on our journey into concurrency with a cautionary tale before we meet some basic theory.

5.1.1 But I already know about Thread

It's one of the most common (and potentially deadly) mistakes a developer can make: to assume that an acquaintance with `Thread`, `Runnable`, and the language-level basic primitives of Java's concurrency mechanism are enough to be a competent developer of concurrent code. In fact, the subject of concurrency is a large one, and good multithreaded development is difficult and continues to cause problems for even the best developers with years of experience under their belts.

It is also true that the area of concurrency is undergoing a massive amount of active research at present—this has been going on for at least the last 5–10 years and shows no signs of abating. These innovations are likely to have an impact on Java and the other languages you'll use over the course of your career.

In the first edition of this book, we made the following claim: "If we were to pick one fundamental area of computing that's likely to change radically in terms of industry practice over the next five years, it would be concurrency." Not only has history borne out this claim, but we feel comfortable rolling this prediction forward—the next five years will see a continued emphasis on the different approaches to concurrency that are now part of the programming landscape.

So, rather than try to be a definitive guide to every aspect of concurrent programming, the aim of this chapter is to make you aware of the underlying platform mechanisms that explain why Java's concurrency works the way it does. We'll also cover enough general concurrency theory to give you the vocabulary to understand the issues involved and to teach you about both the necessity and the difficulty involved in getting concurrency right. First, we'll discuss what every well-grounded Java developer should know about hardware and one of the most important theoretical limitations of concurrency.

1.2 Hardware

Let's start with some basic facts about concurrency and multithreading:

- Concurrent programming is fundamentally about performance.
- There are basically no good reasons for implementing a concurrent algorithm if the system you are running on has sufficient performance that a serial algorithm will work.
- Modern computer systems have multiple processing cores—even mobile phones have two or four cores today.
- All Java programs are multithreaded, even those that have only a single application thread.

This last point is true because the JVM is itself a multithreaded binary that can use multiple cores (e.g., for JIT compilation or garbage collection). In addition, the standard library also includes APIs that use *runtime-managed concurrency* to implement multithreaded algorithms for some execution tasks.

NOTE It is entirely possible that a Java application will run *faster* just by upgrading the JVM it runs on, due to performance improvements in the runtime.

A fuller discussion of hardware takes place in chapter 7, but these basic facts are so fundamental and so relevant to concurrent programming that we want to introduce them immediately.

Now let's meet *Amdahl's law*, named after an early IBM computer scientist, Gene Amdahl, sometimes called the "father of the mainframe."

1.3 Amdahl's law

This is a simple, rough-and-ready model for reasoning about the efficiency of sharing work over multiple execution units. In the model, the execution units are abstract, so

you can think of them as threads, but they could also be processes, or any other entity that is capable of carrying out work.

NOTE None of the setup for or consequences of Amdahl's law depend on the details of how the work is done or the precise nature of the execution units or how the computing systems are implemented.

The basic premise is that we have a single task that can be subdivided into smaller units for processing. This allows us to use multiple execution units to speed up the time taken to complete the work.

So, if we have `N` processors (or threads to do the work), then we might naively expect the elapsed time to be `T1 / N` (if `T1` is the time the job would take on a single processor). In this model, we can finish the job as quickly as we like by just adding execution units and thereby increasing `N`.

However, splitting up the work is not free! A (hopefully small) overhead is involved in the subdividing and recombination of the task. Let's assume that this *communication overhead* (sometime called the *serial part* of the calculation) is an overhead that amounts to a few percent, and we can represent it by a number `s` (0 < `s` < 1). So, a typical value for `s` might be `0.05` (or 5%, whichever way you'd prefer to express it). This means that the task will always take at least `s * T1` to complete—no matter how many processing units we throw at it.

This assumes that `s` does not depend upon `N`, of course, but in practice, the dividing up of work that `s` represents may get more complex and require *more* time as `N` increases. It is extremely difficult to conceive of a system architecture in which `s` *decreases* as `N` increases. So the simple assumption of "`s` is constant" is usually understood to be a *best-case* scenario.

So, the easiest way to think about Amdhal's law is: if `s` is between 0 and 1, then the maximum speedup that can be achieved is `1 / s`. This result is somewhat depressing—it means that if the communication overhead is just 2%, the maximum speedup that can ever be achieved (even with thousands of processors working at full speed) is 50X.

Amdahl's law has a slightly more complex formulation, which is represented like this:

```
T(N) = s + (1/N) * (T1 - s)
```

This can be seen visually in figure 5.1. Note that the *x*-axis is a logarithmic scale—the convergence to `1 / s` would be very hard to see in a linear scale representation.

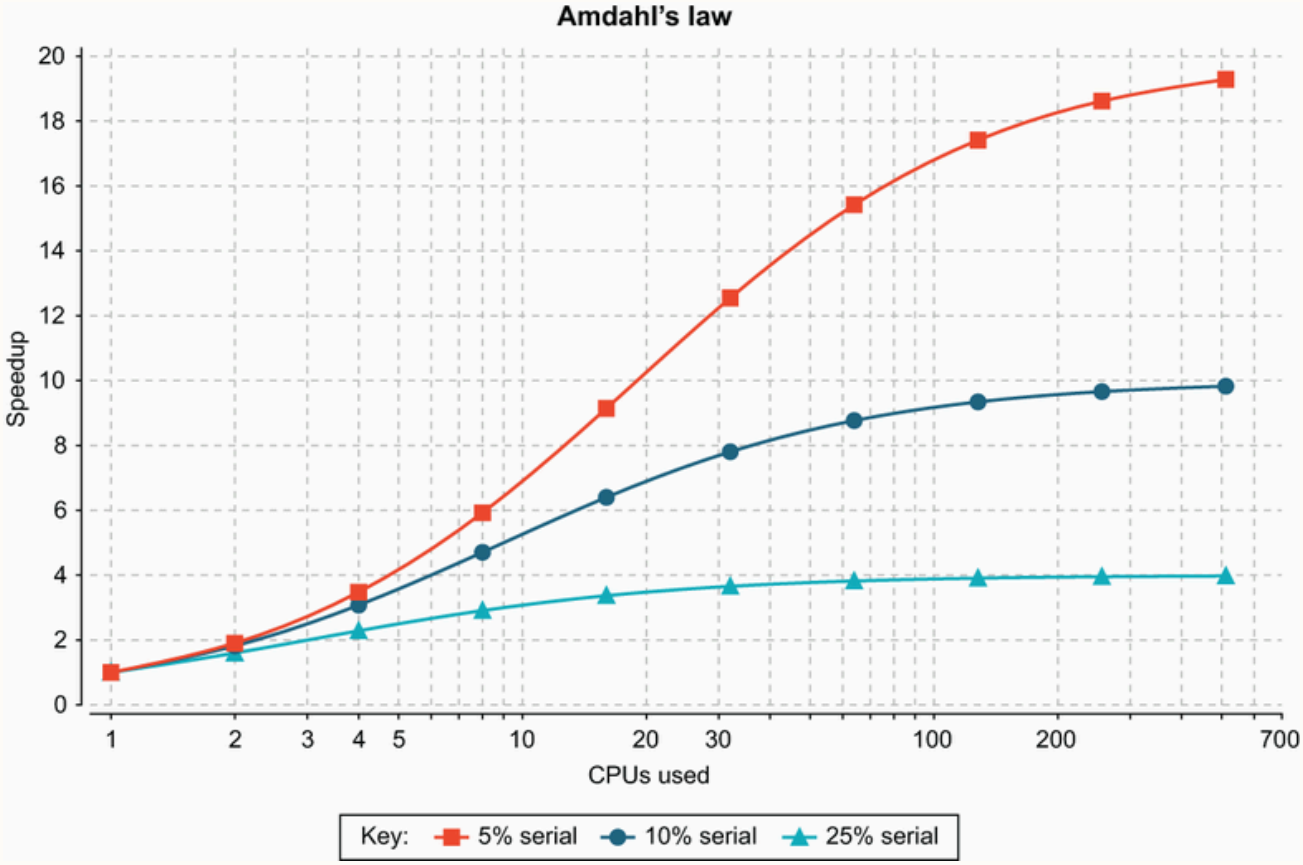


Figure 5.1 Amdahl's law

Having set the scene with hardware and a first, very simple concurrency model, let's dive into the specifics of how Java handles threading.

1.4 Explaining Java's threading model

Java's threading model is based on the following two fundamental concepts:

- Shared, visible-by-default mutable state
- Preemptive thread scheduling by the operating system

Let's consider the following most important aspects of these ideas:

- Objects can be easily shared between all threads within a process.
- Objects can be changed ("mutated") by any threads that have a reference to them.
- The thread scheduler (the operating system) can swap threads on and off cores at any time, more or less.
- Methods must be able to be swapped out while they're running (otherwise, a method with an infinite loop would steal the CPU forever).

- This, however, runs the risk of an unpredictable thread swap, leaving a method "half-done" and an object in an inconsistent state.
- Objects can be *locked* to protect vulnerable data.

The last point is absolutely crucial—without it there is a huge risk of changes being made in one thread not being seen correctly in other threads. In Java, the ability to lock objects is provided by the `synchronized` keyword in the core language.

NOTE Technically, Java provides *monitors* on each of its objects, which combine a lock (aka mutual exclusion) with the ability to wait for a certain condition to become true.

Java's thread-and-lock-based concurrency is very low level and often hard to work with. To cope with this, a set of concurrency libraries, known as `java.util.concurrent` after the Java package where the new classes live, was introduced in Java 5. This provided a set of tools for writing concurrent code that many programmers find easier to use than the classic block-structured concurrency primitives. We will discuss `java .util.concurrent` in the next chapter and will focus on the language-level API for now.

1.5 Lessons learned

Java was the first mainstream programming language to have built-in support for multi-threaded programming. This represented a huge step forward at the time, but now, 15 years later, we've learned a lot more about how to write concurrent code.

It turns out that some of Java's initial design decisions are quite difficult for most programmers to work with. This is unfortunate, because the increasing trend in hardware is toward processors with many cores, and the only good way to take advantage of those cores is with concurrent code. We'll discuss some of the difficulties of concurrent code in this chapter. The subject of modern processors naturally requiring concurrent programming is covered in some detail in chapter 7 where we discuss performance.

As developers become more experienced with writing concurrent code, they find themselves running up against recurring concerns that are important to their systems. We call these concerns *design forces*. They're high-level concepts that exist (and often conflict) in the design of practical concurrent OO systems. We're going to spend a little bit of time looking at some of the most important of these forces in the next couple of sections.

5.2 Design concepts

The most important design forces, listed next, were catalogued by Doug Lea as he was doing his landmark work producing `java.util.concurrent`:

- Safety (also known as *concurrent type safety*)
- Liveness
- Performance
- Reusability

Let's look at each of these forces now.

2.1 Safety and concurrent type safety

Safety is about ensuring that object instances remain self-consistent, regardless of any other operations that may be happening at the same time. If a system of objects has this property, it's said to be *safe* or *concurrently typesafe*.

As you might guess from the name, one way to think about concurrency is in terms of an extension to the regular concepts of object modeling and type safety. In nonconcurrent code, you want to ensure that regardless of what public methods you call on an object, it's in a well-defined and consistent state at the end of the method. The usual way to do this is to keep all of an object's state private and expose a public API of methods that alter the object's state only in a way that makes senses for the design domain.

Concurrent type safety is the same basic concept as type safety for an object, but applied to the much more complex world in which other threads are potentially operating on the same objects on different CPU cores at the same time. For example, consider this simple class:

```
public class StringStack {
    private String[] values = new String[16];
    private int current = 0;

    public boolean push(String s) {
        // Exception handling elided
        if (current < values.length) {
            values[current] = s;
            current = current + 1;
        }
        return false;
    }
```

```
    public String pop() {
      if (current < 1) {
        return null;
      }
      current = current - 1;
      return values[current];
    }
  }
```

When used by single-threaded client code, this is fine. However, preemptive thread scheduling can cause problems. For example, a context switch between execution threads can occur at this point in the code:

```
public boolean push(String s) {
      if (current < values.length) {
        values[current] = s;
        // .... context switch here      (1)
        current = current + 1;
      }
      return false;
    }
```

(1) The object is left in an inconsistent and incorrect state.

If the object is then viewed from another thread, one part of the state (`values`) will have been updated but the other (`current`) will not. Exploring, and solving, this problem is the primary theme of this chapter.

In general, one strategy for safety is to never return from a nonprivate method in an inconsistent state, and to never call any nonprivate method (and certainly not a method on any other object) while in an inconsistent state. If this practice is combined with a way of protecting the object (such as a synchronization lock or critical section) while it's inconsistent, the system can be guaranteed to be safe.

2.2 Liveness

A live system is one in which every attempted activity eventually either progresses or fails. A system that is not live is basically stuck—it will neither progress toward success or fail.

The keyword in the definition is *eventually*—there is a distinction between a transient failure to progress (which isn't that bad in isolation, even if it's not ideal) and a permanent failure. Transient failures could be caused by a number of underlying problems, such as

- Locking or waiting to acquire a lock
- Waiting for input (such as network I/O)
- Temporary failure of a resource
- Not enough CPU time available to run the thread

Permanent failures could be due to a number of causes. Some of the most common follow:

- Deadlock
- Unrecoverable resource problem (such as if the network filesystem [NFS] goes away)
- Missed signal

We'll discuss locking and several of these other problems later in the chapter, although you may already be familiar with some or all of them.

2.3 Performance

The performance of a system can be quantified in a number of different ways. In chapter 7, we'll talk about performance analysis and techniques for tuning, and we'll introduce a number of other metrics you should know about. For now, think of performance as being a measure of how much work a system can do with a given amount of resources.

2.4 Reusability

Reusability forms a fourth design force, because it isn't really covered by any of the other considerations. A concurrent system that has been designed for easy reuse is sometimes very desirable, although this isn't always easy to implement. One approach is to use a reusable toolbox (like `java.util.concurrent`) and build nonreusable application code on top of it.

2.5 How and why do the forces conflict?

The design forces are often in opposition to each other, and this tension can be viewed as a central reason that designing good concurrent systems is difficult, as explained by the following points:

- Safety stands in opposition to liveness—safety is about ensuring that bad things don't happen, whereas liveness requires progress to be made.
- Reusable systems tend to expose their internals, which can cause problems with safety.
- A naïvely written safe system will typically not be very performant, because it usually resorts to heavy use of locking to provide safety guarantees.

The balance that you should ultimately try to achieve is for the code to be flexible enough to be useful for a wide range of problems, closed enough to be safe, and still reasonably live and performant. This is quite a tall order, but, fortunately, some practical techniques can help with this. Here are some of the most common in rough order of usefulness:

1. Restrict the external communication of each subsystem as much as possible. Data hiding is a powerful tool for aiding with safety.
2. Make the internal structure of each subsystem as deterministic as possible. For example, design in static knowledge of the threads and objects in each subsystem, even if the subsystems will interact in a concurrent, nondeterministic way.
3. Apply policy approaches that client apps must adhere to. This technique is powerful but relies on user apps cooperating, and it can be hard to debug if a badly behaved app disobeys the rules.
4. Document the required behavior. This is the weakest of the alternatives, but it's sometimes necessary if the code is to be deployed in a very general context.

The developer should be aware of each of these possible safety mechanisms and should use the strongest possible technique, while being aware that in some circumstances, only the weaker mechanisms are possible.

2.6 Sources of overhead

Many aspects of a concurrent system can contribute to the inherent overhead:

- Monitors (i.e., locks and condition variables)
- Number of context switches
- Number of threads
- Scheduling
- Locality of memory
- Algorithm design

This should form the basis of a checklist in your mind. When developing concurrent code, you should ensure that you have thought about everything on this list.

In particular, the last of these—algorithm design—is an area in which developers can really distinguish themselves, because learning about algorithm design will make you a better programmer in any language.

Two standard texts (highly recommended by the authors) are *Introduction to Algorithms* by Cormen et al. (MIT, 2009)—don't be deceived by the title; this is a serious work—and *The Algorithm Design Manual* (3rd ed.), by Skiena (Springer-Verlag, 2020). For both single-threaded and concurrent algorithms, these books are excellent choices for further reading.

We'll mention many of these sources of overhead in this chapter and the subsequent ones (especially chapter 7, about performance), but now let's turn to our next subject: a review of Java's "classic" concurrency and a close look at why programming with it can be difficult.

5.3 Block-structured concurrency (pre-Java 5)

Much of our coverage of Java concurrency is about discussing alternatives to the language-level, aka block-synchronization-based, aka *intrinsic* approach to concurrency. But to get the most out of the discussion of the alternatives, it's important to have a firm grasp of what's good and bad about the classic view of concurrency.

To that end, for the rest of this chapter, we'll discuss the original, quite low-level way of tackling multithreaded programming using Java's concurrency keywords—`synchronized`, `volatile`, and so on. This discussion will take place in the context of the design forces and with an eye to what will come later on.

Following on from that, we'll briefly consider the life cycle of a thread and then discuss common techniques (and pitfalls) of concurrent code, such as fully synchronized objects, deadlocks, the volatile keyword, and immutability. Let's get started with an overview of synchronization.

5.3.1 Synchronization and locks

As you probably already know, the `synchronized` keyword can be applied either to a block or to a method. It indicates that before entering the block or method, a thread must acquire the appropriate lock. For example, let's think about a method to withdraw money from a bank account, as shown next:

```
public synchronized boolean withdraw(int amount) {    (1)
  // Check to see amount > 0, throw if not
```

```
    if (balance >= amount) {
      balance = balance - amount;
      return true;
    }

    return false;
  }
```

❶ Only one thread can try to withdraw from this account at once.

The method must acquire the lock belonging to the object instance (or the lock belonging to the `Class` object for `static synchronized` methods). For a block, the programmer should indicate which object's lock is to be acquired.

Only one thread can be progressing through any of an object's synchronized blocks or methods at once; if other threads try to enter, they're suspended by the JVM. This is true regardless of whether the other thread is trying to enter the same or a different synchronized block on the same object. In concurrency theory, this type of construct is sometimes referred to as a *critical section*, but this term is more commonly used in C++ than in Java.

NOTE Have you ever wondered why the Java keyword used for a critical section is `synchronized`? Why not "critical" or "locked"? What is it that's being *synchronized*? We'll return to this in section 5.3.5, but if you don't know or have never thought about it, you may want to take a couple of minutes to ponder it before continuing.

Let's look at some basic facts about synchronization and locks in Java. Hopefully you already have most (or all) of these at your fingertips:

- Only objects—not primitives—can be locked.
- Locking an array of objects doesn't lock the individual objects.
- A synchronized method can be thought of as equivalent to a `synchronized (this) { ... }` block that covers the entire method (but note that they're represented differently in bytecode).
- A `static synchronized` method locks the `Class` object, because there's no instance object to lock.
- If you need to lock a `Class` object, consider carefully whether you need to do so explicitly or by using `getClass()`, because the behavior of the two approaches will be different in a subclass.
- Synchronization in an inner class is independent of the outer class (to see why this is so, remember how inner classes are implemented).

- `synchronized` doesn't form part of the method signature, so it can't appear on a method declaration in an interface.
- Unsynchronized methods don't look at or care about the state of any locks, and they can progress while synchronized methods are running.
- Java's locks are reentrant—a thread holding a lock that encounters a synchronization point for the same lock (such as a `synchronized` method calling another `synchronized` method on the same object) will be allowed to continue.

NOTE Non-reentrant locking schemes do exist in other languages (and can be synthesized in Java—see the detail of the Javadoc for `ReentrantLock` in `java.util.concurrent.locks` if you want the gory details), but they're generally painful to deal with, and they're best avoided unless you really know what you're doing.

That's enough review of Java's synchronization. Now let's move on to discuss the states that a thread moves through during its life cycle.

3.2 The state model for a thread

In figure 5.2, you can see the state model for a Java thread. This governs how a Java thread progresses through its life cycle.

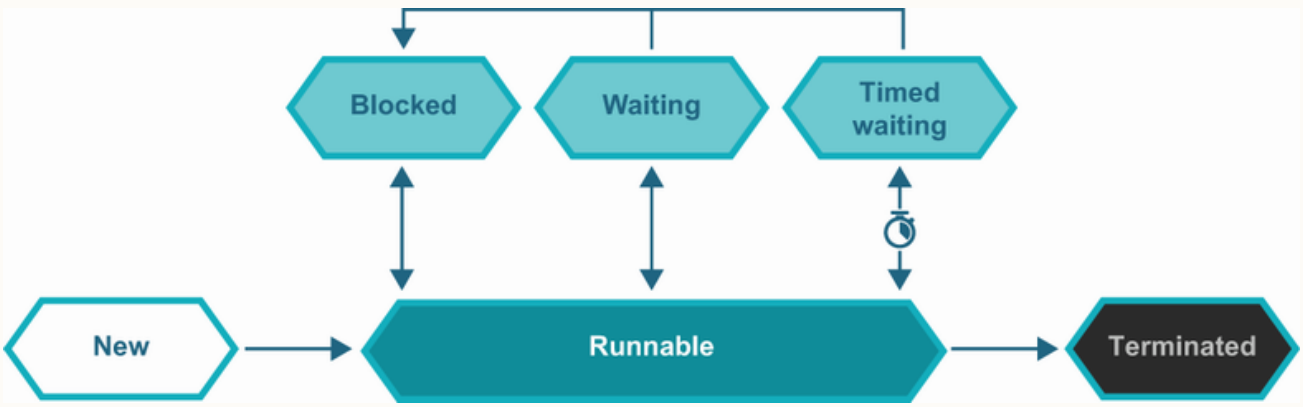


Figure 5.2 The state model of a Java thread

Java has an enum called `Thread.State`, which corresponds to the states in the above state mode and is a layer over the operating system's view of the thread state.

NOTE Every operating system has its own version of threads, and they may differ in the precise details. In most cases, modern operating systems have reasonably similar thread and scheduling implementations, but this was not always the case (e.g., Solaris or Windows XP).

A Java thread object is initially created in the `NEW` state. At this time, an OS thread does not yet exist (and may never exist). To create the execution thread, `Thread.start()` must be called. This signals the OS to actually create a thread.

The scheduler will place the new thread into the run queue and, at some later point, will find a core for it to run upon (some amount of waiting time may be involved if the machine is heavily loaded). From there, the thread can proceed by consuming its time allocation and be placed back into the run queue to await further processor time slices. This is the action of the forcible thread scheduling that we mentioned in section 5.1.1.

Throughout this scheduling process, of being placed on a core, running, and being placed back in the run queue, the Java `Thread` object remains in the `RUNNABLE` state. As well as this scheduling action, the thread itself can indicate that it isn't able to make use of the core at this time. This can be achieved in two different ways:

1. The program code indicates by calling `Thread.sleep()` that the thread should wait for a fixed time before continuing.
2. The thread recognizes that it must wait until some external condition has been met and calls `Object.wait()`.

In both cases, the thread is immediately removed from the core by the OS. However, the behavior after that point is different in each case.

In the first case, the thread is asking to sleep for a definite amount of time. The Java thread transitions into the `TIMED_WAITING` state, and the operating system sets a timer. When it expires, the sleeping thread is woken up and is ready to run again and is placed back in the run queue.

The second case is slightly different. It uses the condition aspect of Java's per-object monitors. The thread will transition into `WAITING` and will wait indefinitely. It will not normally wake up until the operating system signals that the condition may have been met—usually by some other thread calling `Object.notify()` on the current object.

As well as these two possibilities that are under the threads control, a thread can transition into the `BLOCKED` state because it's waiting on I/O or to acquire a lock held by another thread. Finally, if the OS thread corresponding to a Java `Thread` has ceased execution, then that thread object will have transitioned into the `TERMINATED` state. Let's move on to talk about one well-known way to solve the synchronization problem: the idea of fully synchronized objects.

3.3 Fully synchronized objects

Earlier in this chapter, we introduced the concept of concurrent type safety and mentioned one strategy for achieving this. Let's look at a more complete description of this strategy, which is usually called *fully synchronized objects*. If all of the following rules are obeyed, the class is known to be thread-safe and will also be live.

A fully synchronized class is a class that meets all of the following conditions:

- All fields are always initialized to a consistent state in every constructor.
- There are no public fields.
- Object instances are guaranteed to be consistent after returning from any nonprivate method (assuming the state was consistent when the method was called).
- All methods provably terminate in bounded time.
- All methods are synchronized.
- No method calls another instance's methods while in an inconsistent state.
- No method calls any nonprivate method on the current instance while in an inconsistent state.

Listing 5.1 shows an example of such a class from the backend of a banking system. The class `FSOAccount` models an account. The FSO prefix is there to clearly indicate that this implementation uses fully synchronized objects.

This situation provides deposits, withdrawals, and balance queries—a classic conflict between read and write operations—so synchronization is used to prevent inconsistency.

Listing 5.1 A fully synchronized class

```
public class FSOAccount {
   private double balance;                          ❶

   public FSOAccount(double openingBalance) {
      // Check to see openingBalance > 0, throw if not
      balance = openingBalance;                     ❷
   }

   public synchronized boolean withdraw(int amount) {      ❸
      // Check to see amount > 0, throw if not
      if (balance >= amount) {
         balance = balance - amount;
         return true;
      }
```

```
        return false;
    }

    public synchronized void deposit(int amount) {          ❸
        // Check to see amount > 0, throw if not
        balance = balance + amount;
    }

    public synchronized double getBalance() {               ❸
        return balance;
    }
}
```

❶ No public fields

❷ All fields are initialized in the constructor.

❸ All methods are synchronized.

This seems fantastic at first glance—the class is both safe and live. The problem comes with performance. Just because something is safe and live doesn't mean it's necessarily going to be very quick. You have to use `synchronized` to coordinate all the accesses (both get and put) to the balance, and that locking is ultimately going to slow you down. This is a central problem of this way of handling concurrency.

In addition to the performance problems, the code in listing 5.1 is quite fragile. You can see that you never touch `balance` outside of a synchronized method, but this is possible only to check by eye due to the small amount of code in play.

In real, larger systems, this sort of manual verification would not be possible due to the amount of code. It's too easy for bugs to creep into larger codebases that use this approach, which is another reason that the Java community began to look for more robust approaches.

3.4 Deadlocks

Another classic problem of concurrency (and not just Java's take on it) is the *deadlock*. Consider listing 5.2, which is a slightly extended form of the previous example. In this version, as well as modeling the account balance, we also have a `transferTo()` method that can move money from one account to another.

NOTE This is a naïve attempt to build a multithreaded transaction system. It's designed to demonstrate deadlocking—you shouldn't use this as the basis for real code.

In the next listing, let's add a method to transfer funds between two `FSOAccount` objects, like this.

Listing 5.2 A deadlocking example

```
public synchronized boolean transferTo(FSOAccount other, int amount) {
      // Check to see amount > 0, throw if not
      // Simulate some other checks that need to occur
      try {
         Thread.sleep(10);
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      }
      if (balance >= amount) {
         balance = balance - amount;
         other.deposit(amount);
         return true;
      }

      return false;
   }
```

Now, let's actually introduce some concurrency in a main class:

```
public class FSOMain {
   private static final int MAX_TRANSFERS = 1_000;

   public static void main(String[] args) throws InterruptedException {
      FSOAccount a = new FSOAccount(10_000);
      FSOAccount b = new FSOAccount(10_000);
      Thread tA = new Thread(() -> {
         for (int i = 0; i < MAX_TRANSFERS; i = i + 1) {
            boolean ok = a.transferTo(b, 1);
            if (!ok) {
               System.out.println("Thread A failed at "+ i);
            }
         }
      });
      Thread tB = new Thread(() -> {
         for (int i = 0; i < MAX_TRANSFERS; i = i + 1) {
```

```
                boolean ok = b.transferTo(a, 1);
                if (!ok) {
                    System.out.println("Thread B failed at "+ i);
                }
            }
        });
        tA.start();
        tB.start();
        tA.join();
        tB.join();

        System.out.println("End: "+ a.getBalance() + " : "+ b.getBalance());
    }
}
```

At first glance, this code looks sensible. You have two transactions being performed by separate threads. This doesn't seem too outlandish a design— just threads sending money between the two accounts—and all the methods are `synchronized`.

Note that we've introduced a small sleep into the `transferTo()` method. This is to allow the thread scheduler to run both threads and lead to the possibility of deadlock.

NOTE The sleep is for demonstration purposes, not because it is something you'd actually do when writing code for a bank transfer. It's there to simulate code that would actually be there in practice—a delay caused by a call to a database or an authorization check.

If you run the code, you'll normally see an example of a deadlock—both threads will run for a bit and eventually get stuck. The reason is that each thread requires the other to release the lock it holds before the transfer method can progress. This can be seen in figure 5.3.

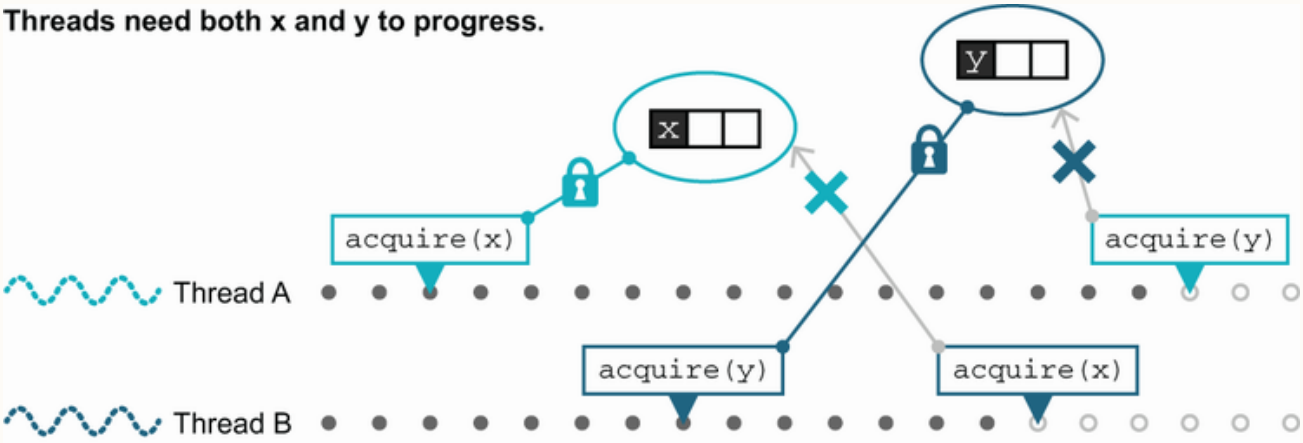


Figure 5.3 Deadlocked threads

Another way of looking at this can be seen in figure 5.4, where we show the Thread Dump view from the JDK Mission Control tool (we will have more to say about this tool in chapter 7 and will show you how to find this useful view then).

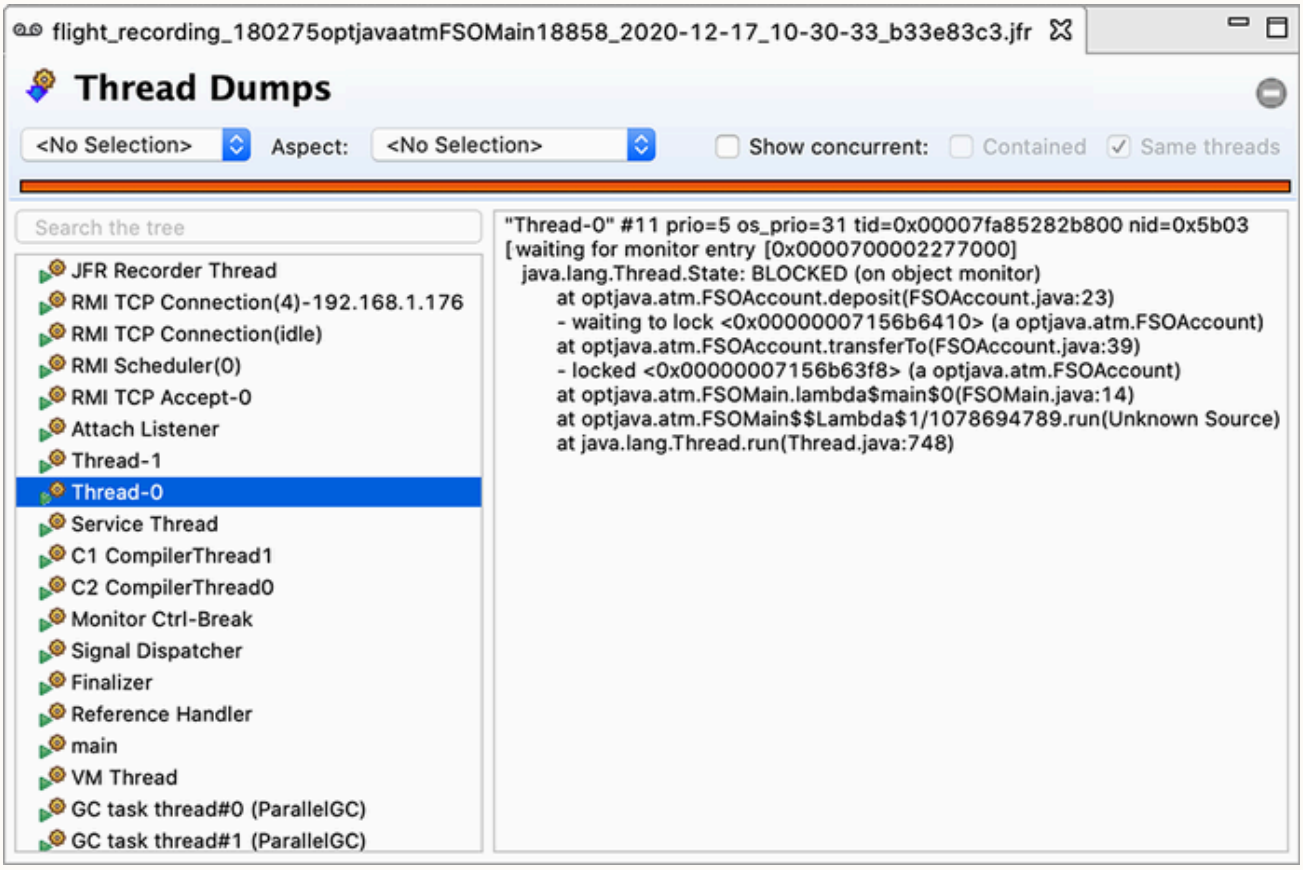


Figure 5.4 Deadlocked threads

The two threads have been created as Thread-0 and Thread-1, and we can see that Thread-0 has locked one reference and is `BLOCKED`, waiting to lock the other. The corresponding thread dump for Thread-1 would show the opposite configuration of the locks, hence the deadlock.

NOTE In terms of the fully synchronized object approach, this deadlock occurs because of the violation of the "bounded time" principle. When the code calls `other.deposit()`, we cannot guarantee how long the code will run for, because the Java Memory Model gives us no guarantees on when a blocked monitor will be released.

To deal with deadlocks, one technique is to always acquire locks in the same order in every thread. In the preceding example, the first thread to start acquires them in the order `A`, `B`, whereas the second thread acquires them in the order `B`, `A`. If both threads had insisted on acquiring them in the order `A`, `B`, the deadlock would have been avoided, because the second thread would have been blocked from running at all until the first had completed and released its locks. Later in the chapter, we will show a simple way to

arrange for all locks to be obtained in the same order and a way to verify that this is indeed satisfied.

Next, we'll return to a puzzle we posed earlier: why the Java keyword for a critical section is named `synchronized`. This will then lead us into a discussion of the `volatile` keyword.

3.5 Why synchronized?

A simple conceptual model of concurrent programming is timesharing of a CPU— that is, threads swapping on and off a single core. This classic view is shown in figure 5.5.

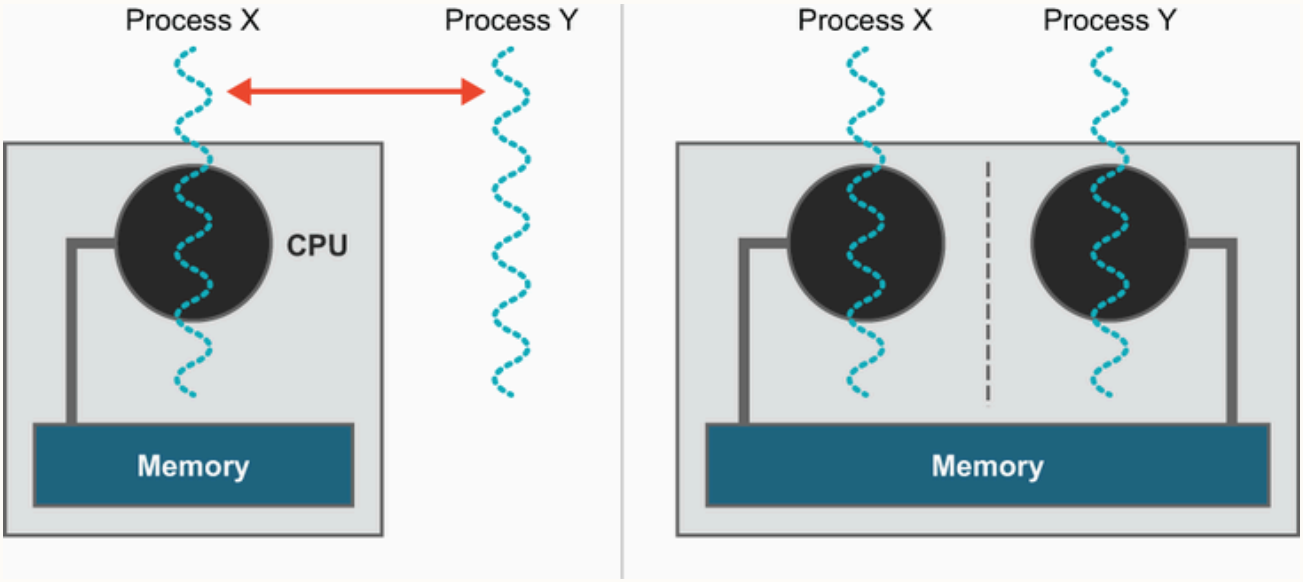


Figure 5.5 The single (left) and multicore (right) of thinking about concurrency and threads

However, this has not been an accurate picture of modern hardware for many years now. Twenty years ago, a working programmer could go for years on end without encountering a system that had more than one or at most two processing cores. That is no longer the case.

Today, anything as big as or larger than a mobile phone has multiple cores, so the mental model should be different, too, encompassing multiple threads all running on different cores at the same physical moment (and potentially operating on shared data). You can see this in figure 5.5. For efficiency, each thread that is running simultaneously may have its own cached copy of data being operated on.

NOTE We will still present theoretical models of execution where our hypothetical computer has only one core. This is purely so that you can see that the nondeterministic concurrency problems we are discussing are inherent and not caused by particular aspects of hardware design.

With this picture in mind, let's turn to the question of the choice of keyword used to denote a locked section or a method.

We asked earlier, what is it that's being synchronized in the code in listing 5.1? The answer is: *the memory representation in different threads of the object being locked.* That is, after the synchronized method (or block) has completed, any and all changes that were made to the object being locked are flushed back to main memory before the lock is released, as illustrated in figure 5.6.

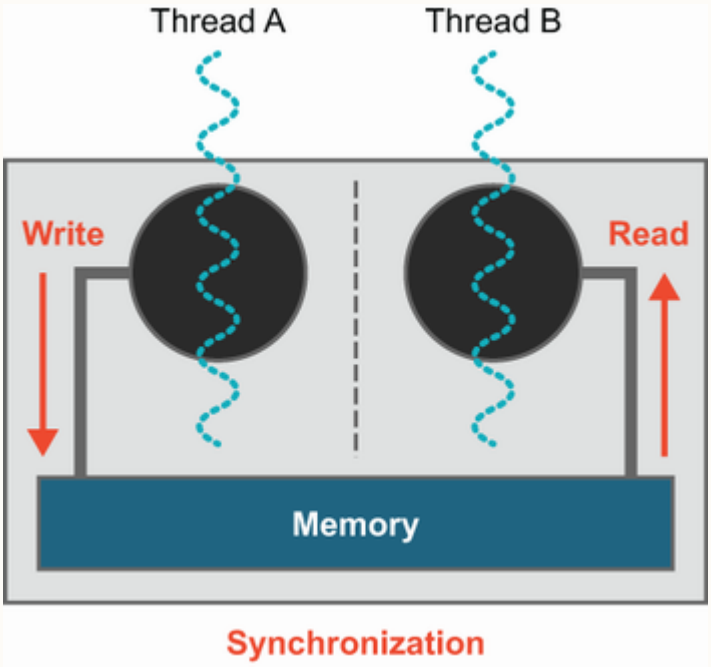


Figure 5.6 A change to an object propagates between threads via the main memory.

In addition, when a synchronized block is entered, after the lock has been acquired, any changes to the locked object are read in *from* main memory, so the thread with the lock is synchronized to the main memory's view of the object before the code in the locked section begins to execute.

3.6 The volatile keyword

Java has had the `volatile` keyword since the dawn of time (Java 1.0), and it's used as a simple way to deal with concurrent handling of object fields, including primitives. The following rules govern a volatile field:

- The value seen by a thread is always reread from the main memory before use.
- Any value written by a thread is always flushed through to the main memory before the bytecode instruction completes.

This is sometimes described as being "like a tiny little synchronized block" around the single operation, but this is misleading because `volatile` does not involve any locking. The action of `synchronized` is to use a mutual exclusion lock on an object to ensure that only one thread can execute a synchronized method on that object. Synchronized methods can contain many read and write operations on the object, and they will be executed as an indivisible unit (from the point of view of other threads) because the results of the method executing on the object are not seen until the method exits and the object is flushed back to main memory.

The key point about `volatile` is that it allows for only *one* operation on the memory location, which will be immediately flushed to memory. This means either a single read, *or* a single write, but not more than that. We saw these two sorts of operations in figure 5.6.

A volatile variable should be used to model a variable only where writes to the variable don't depend on the current state (the read state) of the variable. This is a consequence of `volatile` guaranteeing only a single operation.

For example, the `++` and `--` operators are not safe to use on a `volatile`, because they are equivalent to `v = v + 1` or `v = v - 1`. The increment example is a classic example of a *state-dependent update*.

For cases where the current state matters, you must always introduce a lock to be completely safe. So, `volatile` allows the programmer to write simplified code in some cases, but at the cost of the extra flushes on every access. Notice also that because the volatile mechanism doesn't introduce any locks, you can't deadlock using volatiles—only with synchronization. Later in the chapter, we will meet some other applications for `volatile` and discuss the mechanism in more detail.

3.7 Thread states and methods

A `java.lang.Thread` object is just that: a Java object that lives in the heap and contains metadata about an operating system thread that either exists, used to exist, or will potentially exist in the future.

Java defines the following states for a thread object, which correspond to OS thread states on mainstream operating systems. They are closely related to the state model we saw in figure 5.2:

- `NEW` —the `Thread` object has been created, but the actual OS thread has not.
- `RUNNABLE` —The thread is available to run. The OS is responsible for scheduling it.

- `BLOCKED` —The thread is not running; it needs to acquire a lock or is in a system call.
- `WAITING` —The thread is not running; it has called `Object.wait()` or `Thread .join()` .
- `TIMED_WAITING` —The thread is not running; it has called `Thread.sleep()` .
- `TERMINATED` —The thread is not running; it has completed execution.

All threads start in the `NEW` state and finish in the `TERMINATED` state, whether the thread's `run()` method exits normally or throws an exception.

NOTE The Java thread state model does not distinguish between whether a `RUNNABLE` thread is actually physically executing at that precise moment or is waiting (in the *run queue*).

The actual creation of the thread is done by the `start()` method, which calls into native code to actually perform the relevant system calls (e.g., `clone()` on Linux) to create the thread and begin code execution in the thread's `run()` method.

The standard Thread API in Java breaks down into three groups of methods. Rather than include a lot of boilerplate Javadoc descriptions of each method, we will just list them and leave the reader to consult the API docs for more detail.

The first is a group of methods for reading metadata about the thread:

- `getId()`
- `getName()`
- `getState()`
- `getPriority()`
- `isAlive()`
- `isDaemon()`
- `isInterrupted()`

Some of this metadata (such as the thread ID obtained from `getId()`) will be fixed for the lifetime of the thread. Some of it, such as the thread state and interrupted state, will naturally change as the thread runs, and some of it (e.g., the name and daemon status) can be set by the programmer. This leads us to the second group of methods:

- `setDaemon()`
- `setName()`
- `setPriority()`
- `setUncaughtExceptionHandler()`

It is often better for the programmer to configure any appropriate properties for threads before starting them.

Finally, the following set of thread control methods are used to start new threads and interact with other running threads:

- `start()`
- `interrupt()`
- `join()`

Note that `Thread.sleep()` does not appear in this list, because it's a static method that targets only the current thread.

NOTE Some of the thread methods with timeouts (e.g., `Thread.join()` with a timeout parameter) may actually result in the thread being placed into `TIMED_WAITING` instead of `WAITING`.

Let's take a look at an example of how to use the thread methods in a typical life cycle of a simple multithreaded application:

```
Runnable r = () -> {
    var start = System.currentTimeMillis();
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    var thisThread = Thread.currentThread();
    System.out.println(thisThread.getName() +
        " slept for "+ (System.currentTimeMillis() - start));
};

var t = new Thread(r);          ❶
t.setName("Worker");
t.start();                      ❷
Thread.sleep(100);
t.join();                       ❸
System.out.println("Exiting");
```

❶ The thread's metadata object is created.

❷ The operating system creates an actual thread.

❸ The main thread pauses and waits for the worker to exit before continuing.

This is pretty simple stuff: the main thread creates the worker, starts it, then waits for at least 100 ms (to give the scheduler a chance to run) before reaching the `join()` call, which causes it to pause until the worker thread exits. In the meantime, the worker thread completes the sleep, wakes up again, and prints out the message.

NOTE The elapsed time for the sleep will most likely not be exactly 1000 ms. The operating system scheduler is nondeterministic, and so the best guarantee that is offered is that the operating system will attempt to ensure that the thread sleeps for the requested amount of time, unless woken. However, multithreaded programming is often about dealing with unexpected circumstances, as we will see in the next section.

Interrupting threads

When working with threads, it's relatively common to want to safely interrupt the work a thread is doing, and methods are provided for that on the `Thread` object. However, they may not behave as we'd first expect. Let's run the following code that creates a thread, hard at work, and then tries to interrupt it:

```
var t = new Thread(() -> { while (true); });    ❶
t.start();                                      ❶

t.interrupt();                                  ❷
t.join();                                       ❸
```

❶ Creates and starts a new thread that will run forever

❷ Asks the thread to interrupt itself (i.e., stop executing)

❸ Waits in our main thread for the other to complete

If you run this code, you may be surprised to find that our `join()` will block forever. What's happening here is that thread interruption is opt-in—the methods being called in a thread have to explicitly check the interrupt state and respond to it, and our naive `while` loop never makes such a check. We can fix this in our loop by doing the expected check, as follows:

```
var t = new Thread(() -> { while (!Thread.interrupted()); });    ❶
t.start();                                                       ❶
```

```
t.interrupt();
t.join();
```

❶ Checks our current thread's interrupt state instead of looping on true

Now our loop will exit when requested, and our `join()` no longer blocks forever.

It is common for methods in the JDK that are blocking—whether on IO, waiting on a lock, or other scenarios—to check the thread interrupt state. The convention is that these methods will throw `InterruptedException`, a checked exception. This explains why, for instance, `Thread.sleep()` requires you to add the `InterruptedException` to the method signature or handle it.

Let's modify our example from the previous section to see how `Thread.sleep()` behaves when interrupted:

```
Runnable r = () -> {
    var start = System.currentTimeMillis();
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {    ❶
        e.printStackTrace();
    }
    var thisThread = Thread.currentThread();
    System.out.println(thisThread.getName() +
        " slept for "+ (System.currentTimeMillis() - start));
    if (thisThread.isInterrupted()) {
        System.out.println("Thread "+ thisThread.getName() +" interrupted");
    }
};

var t = new Thread(r);
t.setName("Worker");
t.start();                    ❷
Thread.sleep(100);
t.interrupt();                ❸
t.join();
System.out.println("Exiting");
```

❶ Our Runnable must handle the checked InterruptedException. When we interrupt, it prints the stack, and then execution continues from here.

❷ Creates the worker thread

❸ The main thread interrupts the worker and wakes it up.

When we run this code, we see some output like this:

```
java.lang.InterruptedException: sleep interrupted
    at java.base/java.lang.Thread.sleep(Native Method)
    at examples.LifecycleWithInterrupt.lambda$main$0
     (LifecycleWithInterrupt.java:9)
    at java.base/java.lang.Thread.run(Thread.java:832)
Worker slept for 101
Exiting
```

If you look closely, you will see that the message `"Thread Worker interrupted"` does not appear. This reveals a pertinent fact about handling interrupts in our code: checks for the thread's interruption state actually reset the state. The code that throws the standard `InterruptedException` cleared that interrupt, because it's considered "handled" when the exception is thrown.

NOTE We have the following two methods for checking the interrupt state: a static `Thread.interrupted()`, which implicitly looks at the current thread, and an instance level `isInterrupted()` on a thread object. The static version clears the state after checking and is what's expected for use before throwing an `InterruptedException`. The instance method, on the other hand, doesn't alter the state.

If we want to retain the information that our thread was interrupted, we have to handle that directly ourselves. For our simple example where we need the state only later in the thread's code, something like this would work:

```
Runnable r = () -> {
    var start = System.currentTimeMillis();
    var wasInterrupted = false;              ❶
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        wasInterrupted = true;               ❷
        e.printStackTrace();
    }
    var thisThread = Thread.currentThread();
    System.out.println(thisThread.getName() +
        " slept for "+ (System.currentTimeMillis() - start));
```

```
    if (wasInterrupted) {
        System.out.println("Thread "+ thisThread.getName() +" interrupted");
    }
};

var t = new Thread(r);
t.setName("Worker");
t.start();
Thread.sleep(100);
t.interrupt();
t.join();
System.out.println("Exiting");
```

❶ Sets up the state to record a possible interruption

❷ Records the interruption

In more complex situations, you may wish to ensure an `InterruptedException` is rethrown for callers, throw a custom exception of some sort, perform your own custom logic, or even restore the interrupt state onto the thread in question. All of these are possible, depending on your specific needs.

Working with exceptions and threads

Another issue for multithreaded programming is how to handle exceptions that may be thrown from within a thread. For example, suppose that we are executing a `Runnable` of unknown provenance. If it throws an exception and dies, then other code may not be aware of it. Fortunately, the Thread API provides the ability to add uncaught exception handlers to a thread before starting it, like this example:

```
var badThread = new Thread(() -> {
    throw new UnsupportedOperationException(); });

// Set a name before starting the thread
badThread.setName("An Exceptional Thread");

// Set the handler
badThread.setUncaughtExceptionHandler((t, e) -> {
    System.err.printf("Thread %d '%s' has thrown exception " +
                      "%s at line %d of %s",
            t.getId(),
            t.getName(),
            e.toString(),
```

```
            e.getStackTrace()[0].getLineNumber(),
            e.getStackTrace()[0].getFileName()); });

    badThread.start();
```

The handler is an instance of `UncaughtExceptionHandler`, which is a functional interface, defined like this:

```
public interface UncaughtExceptionHandler {
    void uncaughtException(Thread t, Throwable e);
}
```

This method provides a simple callback to allow thread control code to take action based on the observed exception—for example, a thread pool may restart a thread that has exited in this way to maintain pool size.

NOTE Any exception thrown by `uncaughtException()` will be ignored by the JVM.

Before we move on, we need to discuss some other control methods of `Thread` that are deprecated and that should not be used by application programmers.

Deprecated thread methods

Java was the first mainstream language to support multithreaded programming out of the box. However, this "first mover" advantage had its dark side—many of the inherent issues that exist with concurrent programming were first encountered by programmers working in Java.

One aspect of this is the unfortunate fact that some of the methods in the original Thread API are simply unsafe and unfit to use, in particular, `Thread.stop()`. This method is essentially impossible to use safely—it kills another thread without any warning and with no way for the killed thread to ensure that any locked objects are made safe.

The deprecation of `stop()` followed close on the heels of its active use in early Java because stopping another thread required injecting an exception into the other thread's execution. However, it's impossible to know precisely *where* that other thread is in execution. Maybe the thread is killed in the middle of a `finally` block the developer assumed would run fully, and the program is left in a corrupted state.

The mechanism is that an unchecked `ThreadDeath` exception is triggered on the killed thread. It is not feasible for code to guard against such an exception with try blocks (any more than it is possible to reliably protect against an `OutOfMemoryError`), and so the exception immediately starts unwinding the stack of the killed thread, and all monitors are unlocked. This immediately makes potentially damaged objects visible to other threads, and so `stop()` is just not safe for use.

In addition to the reasonably well-known issues with `stop()`, several other methods also have serious issues. For example, `suspend()` does not cause any monitors to be released, so any thread that attempts to access any synchronized code that is locked by the suspended thread will block permanently, unless the suspended thread is reactivated. This represents a major liveness hazard, and so `suspend()` and `resume()` should never be used. The `destroy()` method was never implemented, but it would have suffered from the same issues if it had been.

NOTE These dangerous thread methods have been deprecated since Java 1.2—over 20 years ago—and have recently been marked for removal (which will be a breaking change, to give you some idea of how seriously this is viewed).

The real solution to the problem of reliably controlling threads from other threads is best illustrated by the *Volatile Shutdown* pattern that we will meet later in the chapter. Now let's move on to one of the most useful techniques when handling data that must be shared safely when programming in a concurrent style.

3.8 Immutability

One technique that can be of great value is the use of immutable objects. These are objects that either have no state or that have only final fields (which must, therefore, be populated in the constructors of the objects). These are always safe and live, because their state can't be mutated, so they can never be in an inconsistent state.

One problem is that any values that are required to initialize a particular object must be passed into the constructor. This can lead to unwieldy constructor calls, with many parameters. As a result, many coders use a *factory method* instead. This can be as simple as using a static method on the class, instead of a constructor, to produce new objects. The constructors are usually made protected or private, so that the static factory methods are the only way of instantiating. For example, consider a simple deposit class that we might see in a banking system, as shown here:

```
public final class Deposit {
    private final double amount;
    private final LocalDate date;
    private final Account payee;

    private Deposit(double amount, LocalDate date, Account payee) {
        this.amount = amount;
        this.date = date;
        this.payee = payee;
    }

    public static Deposit of(double amount, LocalDate date, Account payee) {
        return new Deposit(amount, date, payee);
    }

    public static Deposit of(double amount, Account payee) {
        return new Deposit(amount, LocalDate.now(), payee);
    }
```

This has the fields of the class, a private constructor, and two factory methods, one of which is a convenience method for creating deposits for today. Next up are the accessor methods for the fields:

```
    public double amount() {
        return amount;
    }

    public LocalDate date() {
        return date;
    }

    public Account payee() {
        return payee;
    }
```

Note that in our example, these are presented in *record style*, where the name of the accessor method matches the name of the field. This is in contrast to *bean style*, when getter methods are prefixed with `get` and setter methods (for any nonfinal fields) are prefixed with `set`.

Immutable objects obviously can't be changed, so what happens when we want to make changes to one of them? For example, it's very common that if a deposit or other trans-

action can't take place on a specific day, that transaction is "rolled" to the following day. We can achieve this by having an instance method on the type that returns an object that is almost the same, but has some modified fields, as follows:

```
public Deposit roll() {
   // Log audit event for rolling the date
   return new Deposit(amount, date.plusDays(1), payee);
}

public Deposit amend(double newAmount) {
   // Log audit event for amending the amount
   return new Deposit(newAmount, date, payee);
}
```

One problem that immutable objects potentially have is that they may need many parameters to be passed in to the factory method. This isn't always very convenient, especially when you may need to accumulate state from several sources before creating a new immutable object.

To solve this, we can use the Builder pattern. This is a combination of two constructs: a static inner class that implements a generic builder interface, and a private constructor for the immutable class itself.

The static inner class is the builder for the immutable class, and it provides the only way that a developer can get hold of new instances of the immutable type. One very common implementation is for the `Builder` class to have exactly the same fields as the immutable class but to allow mutation of the fields. This listing shows how you might use this to model a more complex view of a deposit.

Listing 5.3 Immutable objects and builders

```
public static class DepositBuilder implements Builder<Deposit> {
   private double amount;
   private LocalDate date;
   private Account payee;

   public DepositBuilder amount(double amount) {
      this.amount = amount;
      return this;
   }

   public DepositBuilder date(LocalDate date) {
```

```
            this.date = date;
            return this;
        }

        public DepositBuilder payee(Account payee) {
            this.payee = payee;
            return this;
        }

        @Override
        public Deposit build() {
            return new Deposit(amount, date, payee);
        }
    }
```

The builder is a generic top-level interface, which is usually defined like this:

```
public interface Builder<T> {
    T build();
}
```

We should note a couple of things about the builder. First of all, it is a so-called SAM type (for "single abstract method"), and it could, technically speaking, be used as the target type for a lambda expression. However, the purpose of the builder is to produce immutable instances—it is about gathering up state, not representing a function or call-back. This means that although the builder *could* be used as a functional interface, in practice, it will never be useful to do so.

For this reason, we do *not* decorate the interface with the `@FunctionalInterface` annotation—another good example of "just because you can do something, doesn't mean that you should."

Secondly, we should also notice that the builder is not thread-safe. The design implicitly assumes that the user knows not to share builders between threads. Instead, correct usage of the Builder API is for one thread to use a builder to aggregate all needed state and then produce an immutable object that *can* be trivially shared with other threads.

NOTE If you find yourself wanting to share a builder between threads, take a moment to stop and reconsider your design and whether your domain needs refactoring.

Immutability is a very common pattern (not just in Java, but also in other languages, especially functional languages) and is one that has wide applicability.

One last point about immutable objects: the `final` keyword applies only to the object directly pointed to. As you can see in figure 5.7, the reference to the main object can't be assigned to point at object 3, but within the object, the reference to object 1 can be updated to point at object 2. Another way of saying this is that a `final` reference can point at an object that has nonfinal fields. This is sometimes known as *shallow immutability*.

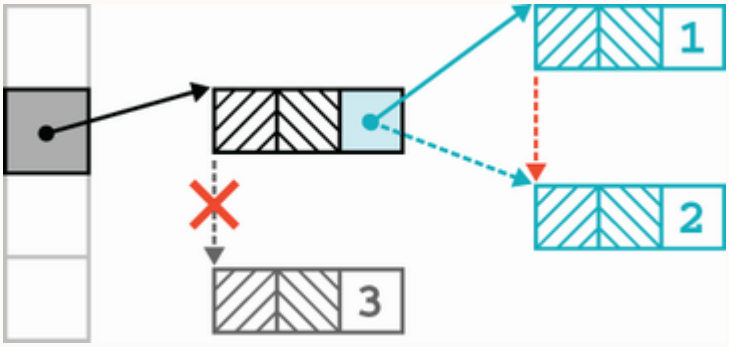


Figure 5.7 Immutability of value versus reference

Another way of seeing this is that it is perfectly possible to write the following:

```
final var numbers = new LinkedList<Integer>();
```

In this statement, the reference `numbers` and the integer objects contained in the list are immutable. However, the list object itself is still mutable because integer objects can still be added, removed, and replaced with the list.

Immutability is a very powerful technique, and you should use it whenever feasible. However, sometimes it's just not possible to develop efficiently with only immutable objects, because every change to an object's state requires a new object to be spun up. So we're sometimes left with the necessity of dealing with mutable objects.

In the next section, we'll discuss the often-misunderstood details of the Java Memory Model (JMM). Many Java programmers are aware of the JMM and have been coding to their own understanding of it without ever being formally introduced to it. If that sounds like you, this new understanding will build upon your informal awareness and place it onto firm foundations. The JMM is quite an advanced topic, so you can skip it if you're in a hurry to get on to the next chapter.

5.4 The Java Memory Model (JMM)

The JMM is described in section 17.4 of the Java Language Specification (JLS). This is a formal part of the spec, and it describes the JMM in terms of synchronization actions and some quite mathematical concepts, for example, a *partial order* for operations.

This is great from the point of view of language theorists and implementers of the Java spec (compiler and JVM makers), but it's worse for application developers who need to understand the details of how their multithreaded code will execute.

Rather than repeat the formal details, we'll list the most important rules here in terms of a couple of basic concepts: the *Synchronizes-With* and *Happens-Before* relationships between blocks of code:

- *Happens-Before*—This relationship indicates that one block of code fully completes before the other can start.
- *Synchronizes-With*—An action will synchronize its view of an object with main memory before continuing.

If you've studied formal approaches to OO programming, you may have heard the expressions *Has-A* and *Is-A* used to describe the building blocks of object orientation. Some developers find it useful to think of Happens-Before and Synchronizes-With as being similar, basic conceptual building blocks, but for understanding Java concurrency rather than OO. However, we should emphasize that no direct technical connection exists between the two sets of concepts. In figure 5.8 you can see an example of a volatile write that Synchronizes-With a later read access (for the `println()`).

The JMM has these main rules:

- An unlock operation on a monitor Synchronizes-With later locks operations.
- A write to a volatile variable Synchronizes-With later reads from the variable.
- If an action A Synchronizes-With action B, then A Happens-Before B.
- If A comes before B in program order, within a thread, then A Happens-Before B.

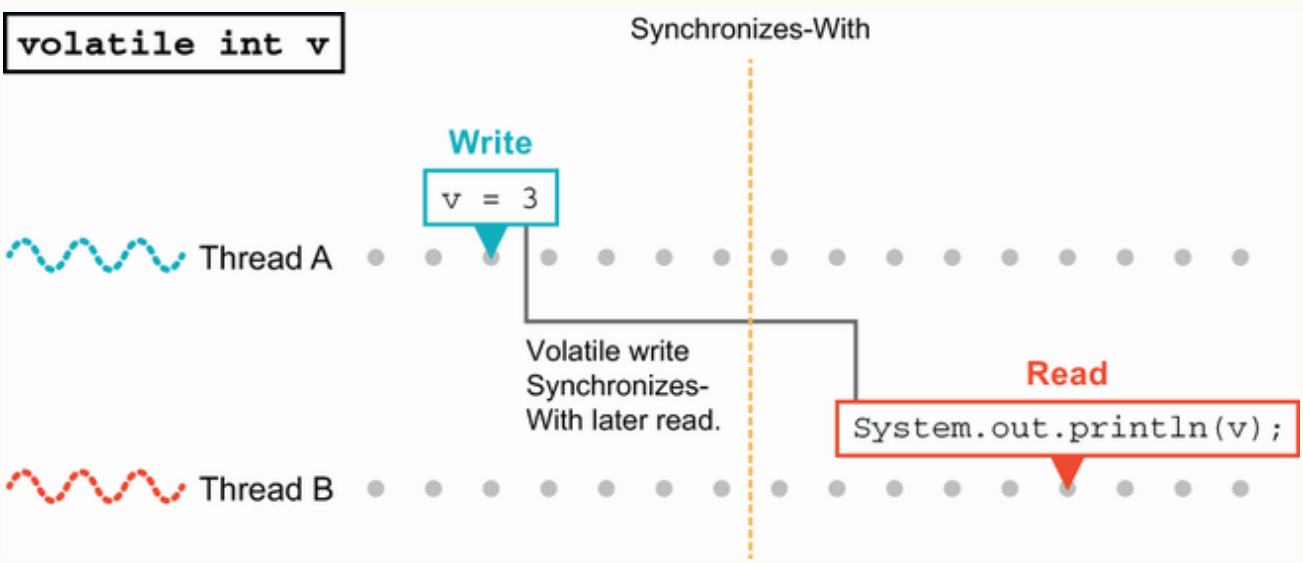


Figure 5.8 A Synchronizes-With example

The general statement of the first two rules is that "releases happen before acquires." In other words, the locks that a thread holds when writing are released before the locks can be acquired by other operations (including reads). For example, the rules guarantee that if one thread writes a value to a volatile variable, then any thread that later reads that variable will see the value that was written (assuming no other writes have taken place).

Additional rules, which are really about sensible behavior, follow:

- The completion of a constructor Happens-Before the finalizer for that object starts to run (an object has to be fully constructed before it can be finalized).
- An action that starts a thread Synchronizes-With the first action of the new thread.
- `Thread.join()` Synchronizes-With the last (and all other) actions in the thread being joined.
- If X Happens-Before Y and Y Happens-Before Z, then X Happens-Before Z (transitivity).

These simple rules define the whole of the platform's view of how memory and synchronization works. Figure 5.9 illustrates the transitivity rule.



Figure 5.9 Transitivity of Happens-Before

NOTE In practice, these rules are the minimum guarantees made by the JMM. Real JVMs may behave much better in practice than these guarantees suggest. This can be quite a pitfall for the developer because it's easy for the false sense of safety given by the behavior of a particular JVM to turn out to be just a quirk hiding an underlying concurrency bug.

From these minimum guarantees, it's easy to see why immutability is an important concept in concurrent Java programming. If objects can't be changed, there are no issues related to ensuring that changes are visible to all threads.

5.5 Understanding concurrency through bytecode

Let's discuss concurrency through the lens of a classic example: a bank account. Let's assume that a customer's account looks like this and that withdrawals and deposits are possible by calling methods. We have provided both synchronized and unsynchronized implementations of the key methods:

```
public class Account {
   private double balance;

   public Account(int openingBalance) {
      balance = openingBalance;
   }

   public boolean rawWithdraw(int amount) {
      // Check to see amount > 0, throw if not
```

```
        if (balance >= amount) {
            balance = balance - amount;
            return true;
        }
        return false;
    }

    public void rawDeposit(int amount) {
        // Check to see amount > 0, throw if not
        balance = balance + amount;
    }

    public double getRawBalance() {
        return balance;
    }

    public boolean safeWithdraw(final int amount) {
        // Check to see amount > 0, throw if not
        synchronized (this) {
            if (balance >= amount) {
                balance = balance - amount;
                return true;
            }
        }
        return false;
    }

    public void safeDeposit(final int amount) {
        // Check to see amount > 0, throw if not
        synchronized (this) {
            balance = balance + amount;
        }
    }

    public double getSafeBalance() {
        synchronized (this) {
            return balance;
        }
    }
}
```

This set of methods will allow us to explore many common concurrency problems in Java.

NOTE There is a reason we are using the block form of synchronization at this stage, rather than the `synchronized` method modifier—we will explain why a bit later in the chapter.

We might also suppose that, if required, the class has two argument helper methods that look like this:

```
public boolean withdraw(int amount, boolean safe) {
  if (safe) {
    return safeWithdraw(amount);
  } else {
    return rawWithdraw(amount);
  }
}
```

Let's start by meeting one of the fundamental problems that multithreaded systems display that requires us to introduce some sort of protection mechanism.

5.1 Lost Update

To demonstrate this common problem (or antipattern), known as *Lost Update*, let's look at the bytecode for the `rawDeposit()` method:

```
public void rawDeposit(int);
  Code:
    0: aload_0
    1: aload_0
    2: getfield      #2  // Field balance:D   (1)
    5: iload_1
    6: i2d
    7: dadd                                  (2)
    8: putfield      #2  // Field balance:D   (3)
   11: return
```

(1) Reads the balance from the object

(2) Adds the deposit amount

(3) Writes the new balance to the object

Let's introduce two threads of execution, called `A` and `B`. We can then imagine two deposits being attempted on the same account at once. By prefixing the instruction with the thread label, we can see individual bytecode instructions executing on different threads, but they are both affecting the same object.

NOTE Remember that some bytecode instructions have parameters that follow them in the stream, which causes occasional "skips" in the instruction numbering.

Lost Update is the issue that it is possible, due to the nondeterministic scheduling of application threads, to end up with a bytecode sequence of reads and writes like this:

```
A0: aload_0
    A1: aload_0
    A2: getfield      #2  // Field balance:D      ❶
    A5: iload_1
    A6: i2d
    A7: dadd

// ....          Context switch A -> B

        B0: aload_0
        B1: aload_0
        B2: getfield      #2  // Field balance:D  ❷
        B5: iload_1
        B6: i2d
        B7: dadd
        B8: putfield      #2  // Field balance:D  ❸
       B11: return

// ....          Context switch B -> A

    A8: putfield      #2  // Field balance:D      ❹
   A11: return
```

❶ Thread A reads a value from the balance.

❷ Thread B reads the same value from the balance as A did.

❸ Thread B writes a new value back to the balance.

❹ Thread A overwrites the balance—B's update is lost.

The updated balance is calculated by each thread by using the evaluation stack. The `dadd` opcode is the point where the updated balance is placed on the stack, but recall that every method invocation has its own, private evaluation stack. So, at point `B7` in the previous flow are *two* copies of the updated balance: one in A's evaluation stack and one in B's. The two `putfield` operations at `B8` and `A8` then execute, but `A8` overwrites the value placed at `B8`. This leads to the situation where both deposits appear to succeed, but only one of them actually shows up.

The account balance will register a deposit, but the code will still cause money to vanish from the account, because the balance field is read twice (with `getfield`), then written and overwritten (by the two `putfield` operations). For example, in some code like this:

```
Account acc = new Account(0);
Thread tA = new Thread(() -> acc.rawDeposit(70));
Thread tB = new Thread(() -> acc.rawDeposit(50));
tA.start();
tB.start();
tA.join();
tB.join();

System.out.println(acc.getRawBalance());
```

it is possible for the final balance to be either 50 or 70—but with both threads "successfully" depositing money. The code has paid in 120 but has lost some of it—a classic example of incorrect multithreaded code.

Be careful with the simple form of the code shown here. The full range of nondeterministic possibilities may not show up in such a simple example. Do not be fooled by this—when this code is combined into a large program, the incorrectness will assuredly appear. Assuming that your code is OK because it's "too simple" or trying to cheat the concurrency model will inevitably end badly.

NOTE There is an example (`AtmLoop`) that shows this effect in the source code repository, but it relies upon using a class we haven't met yet (`AtomicInteger`) so we won't show it in full here. So, if you need to be convinced, please go and examine how the example behaves.

In general, access patterns like

```
A: getfield
B: getfield
```

```
B: putfield
A: putfield
```

or

```
A: getfield
B: getfield
A: putfield
B: putfield
```

will cause problems for our account objects.

Recall that the operating system effectively causes nondeterministic scheduling of threads, so this type of interleaving is always possible, and Java objects live in the heap, so the threads are operating on shared, mutable data.

What we really need is to introduce a mechanism to somehow prevent this and ensure that the ordering is always of the following form:

```
...
A: getfield
A: putfield
...
B: getfield
B: putfield
...
```

This mechanism is synchronization, and it is our next subject.

5.2 Synchronization in bytecode

In chapter 4, we introduced JVM bytecodes and briefly met `monitorenter` and `monitorexit`. A synchronized block is turned into these opcodes (we'll talk about synchronized methods a bit later). Let's see them in action by looking at an example we saw earlier (we're reproducing the Java code so it's close at hand):

```
    public boolean safeWithdraw(final int amount) {
        // Check to see amount > 0, throw if not
        synchronized (this) {
            if (balance >= amount) {
```

```
            balance = balance - amount;
            return true;
          }
        }
        return false;
    }
```

This is turned into 40 bytes of JVM bytecode:

```
public boolean safeWithdraw(int);
  Code:
    0: aload_0
    1: dup
    2: astore_2
    3: monitorenter                    (1)
    4: aload_0
    5: getfield     #2  // Field balance:D
    8: iload_1
    9: i2d
   10: dcmpl
   11: iflt         29                (2)
   14: aload_0
   15: aload_0
   16: getfield     #2  // Field balance:D
   19: iload_1
   20: i2d
   21: dsub
   22: putfield     #2  // Field balance:D   (3)
   25: iconst_1
   26: aload_2
   27: monitorexit                    (4)
   28: ireturn                        (5)
   29: aload_2
   30: monitorexit                    (4)
   31: goto         39
   34: astore_3
   35: aload_2
   36: monitorexit                    (4)
   37: aload_3
   38: athrow
   39: iconst_0
   40: ireturn                        (5)
```

(1) The start of the synchronized block

❷ The if statement that checks the balance

❸ Writes the new value to the balance field

❹ The end of the synchronized block

❺ Returns from the method

The eagle-eyed reader might spot a couple of oddities in the bytecode. First of all, let's look at the code paths. If the balance check succeeds, then bytecodes 0–28 are executed with no jumps. If it fails, bytecodes 0–11 execute, then a jump to 29–31 and a jump to 39–40.

At first glance, no set of circumstances will lead to bytecodes 34–38 being executed. This seeming discrepancy is actually explained by exception handling—some of the bytecode instructions (including `monitorenter`) can throw exceptions, so there needs to be an exception handling code path.

The second puzzler is the return type of the method. In the Java code, it is declared as `boolean`, but we can see that the return instructions are `ireturn`, which is the integer variant of the return opcode. In fact, no variant forms of instructions for bytes, shorts, chars, or booleans exist. These types are replaced by ints during the compilation process. This is a form of *type erasure*, which is one of the misunderstood aspects of Java's type system (especially as it is usually applied to the case of generics and type parameters).

Overall, the previous bytecode sequence is more complex than the nonsynchronized case but should be possible to follow: we load the object to be locked onto the evaluation stack and then execute a `monitorenter` to acquire the lock. Let's assume the lock attempt succeeds.

Now, if any other thread attempts to execute a `monitorenter` on the same object, the thread will block, and the second `monitorenter` instruction will not complete until the thread holding the lock executes a `monitorexit` and releases the lock. This is how we deal with Lost Update—the `monitor` instructions are enforcing the following ordering:

```
...
A: monitorenter
A: getfield
A: putfield
A: monitorexit
...
```

```
B: monitorenter
B: getfield
B: putfield
B: monitorexit
...
```

This provides the Happens-Before relationship between synchronized blocks: the end of one synchronized block Happens-Before the start of any other synchronized block on the same object, and this is guaranteed by the JMM.

We should also note that the Java source compiler ensures that every code path through a method that contains a `monitorenter` will result in a `monitorexit` being executed before the method terminates. Not only this, but at class loading time, the classfile verifier will reject any class that tries to circumvent this rule.

We can now see the basis for the claim that "synchronization is a cooperative mechanism in Java." Let's look at what happens when thread A calls `safeWithdraw()` and thread B calls `rawDeposit()`:

```
public boolean safeWithdraw(final int amount) {
   // Check to see amount > 0, throw if not
   synchronized (this) {
      if (balance >= amount) {
         balance = balance - amount;
         return true;
      }
   }
   return false;
}
```

We've reproduced the Java code once again for easy comparison:

```
public boolean safeWithdraw(int);
   Code:
     0: aload_0
     1: dup
     2: astore_2
     3: monitorenter
     4: aload_0
     5: getfield      #2  // Field balance:D
     8: iload_1
     9: i2d
```

```
      10: dcmpl
      11: iflt        29
      14: aload_0
      15: aload_0
      16: getfield    #2  // Field balance:D
      19: iload_1
      20: i2d
      21: dsub
      22: putfield    #2  // Field balance:D
      25: iconst_1
      26: aload_2
      27: monitorexit
      28: ireturn
```

The depositing code is very simple: just one field read, an arithmetic operation, and a write back to the same field, as shown here:

```
public void rawDeposit(int amount) {
      // Check to see amount > 0, throw  if not
      balance = balance + amount;
   }
```

The bytecode looks more complicated but actually isn't:

```
public void rawDeposit(int);
   Code:
      0: aload_0
      1: aload_0
      2: getfield     #2  // Field balance:D
      5: iload_1
      6: i2d
      7: dadd
      8: putfield     #2  // Field balance:D
     11: return
```

NOTE The code for `rawDeposit()` does not contain any `monitor` instructions—and without a `monitorenter`, the lock will never be checked.

An ordering like this, between two threads `A` and `B`, is entirely possible, as shown next:

```
// ...
 A3: monitorenter
// ...

A14: aload_0
A15: aload_0
A16: getfield      #2  // Field balance:D

// ... Context switch A -> B

 B0: aload_0
 B1: aload_0
 B2: getfield      #2  // Field balance:D
 B5: iload_1
 B6: i2d
 B7: dadd
 B8: putfield      #2  // Field balance:D    (1)

// ... Context switch B -> A

B11: return
A19: iload_1
A20: i2d
A21: dsub
A22: putfield      #2  // Field balance:D    (2)
A25: iconst_1
A26: aload_2
A27: monitorexit
A28: ireturn
```

(1) Writes to the balance (via unsynchronized method)

(2) Second write to the balance (via synchronized)

This is just our old friend Lost Update, but now it occurs when one of the methods uses synchronization and one doesn't. The amount deposited has been lost—good news for the bank, but not so good for the customer. The inescapable conclusion is this: to get the protections provided by synchronization, *all* methods must use it correctly.

5.3 Synchronized methods

Until now, we have been talking about the case of synchronized blocks, but what about the case of synchronized *methods*? We might guess that the compiler will insert syn-

thetic `monitor` bytecodes, but that's actually not the case, as we can see if we change our safe methods to look like this:

```
public synchronized boolean safeWithdraw(final int amount) {
   // Check to see amount > 0, throw if not
   if (balance >= amount) {
      balance = balance - amount;
      return true;
   }
   return false;
}

// and the others...
```

Instead of showing up in the bytecode sequence, the `synchronized` modifier for the method actually shows up in the method's flags, as `ACC_SYNCHRONIZED`. We can see this by recompiling the method and noticing that the `monitor` instructions have disappeared, as shown here:

```
public synchronized boolean safeWithdraw(int);
Code:
  0: aload_0
  1: getfield      #2  // Field balance:D
  4: iload_1
  5: i2d
  6: dcmpl
  7: iflt          23
 10: aload_0
 // ... no monitor instructions
```

When executing an `invoke` instruction, one of the first things the bytecode interpreter checks is to see whether the method is `synchronized`. If it is, then the interpreter proceeds down a different code path—first by trying to acquire the appropriate lock. If the method does not have the `ACC_SYNCHRONIZED`, then no such check is done.

This means that, just as we might expect, an unsynchronized method can execute at the same time as a synchronized one because only one of them performs a check for the lock.

5.4 Unsynchronized reads

A very common beginner's error with Java concurrency is to assume that "only methods that write data need to be synchronized; reads are safe." This is emphatically not true, as we will demonstrate.

This false sense of security for reads sometimes occurs because the code example being reasoned about is a bit too simple. What happens when we introduce a small ATM fee to our example—say, 1% of the amount being withdrawn?

```
    private final double atmFeePercent = 0.01;

    public boolean safeWithdraw(final int amount, final boolean withFee) {
       // Check to see amount > 0, throw if not
       synchronized (this) {
          if (balance >= amount) {
             balance = balance - amount;
             if (withFee) {
                balance = balance - amount * atmFeePercent;
             }
             return true;
          }
       }
       return false;
    }
```

The bytecode for this method is now a bit more complex:

```
  public boolean safeWithdraw(int, boolean);
     Code:
       0: aload_0
       1: dup
       2: astore_3
       3: monitorenter
       4: aload_0
       5: getfield      #2  // Field balance:D
       8: iload_1
       9: i2d
      10: dcmpl
      11: iflt         49                    (1)
      14: aload_0
      15: aload_0
      16: getfield      #2  // Field balance:D
```

```
19: iload_1
20: i2d
21: dsub
22: putfield      #2  // Field balance:D      ❷
25: iload_2
26: ifeq          45
29: aload_0
30: aload_0
31: getfield      #2  // Field balance:D
34: iload_1
35: i2d
36: aload_0
37: getfield      #5  // Field atmFeePercent:D
40: dmul
41: dsub
42: putfield      #2  // Field balance:D      ❸
45: iconst_1
46: aload_3
47: monitorexit
48: ireturn
49: aload_3
50: monitorexit
51: goto          61
54: astore        4
56: aload_3
57: monitorexit
58: aload         4
60: athrow
61: iconst_0
62: ireturn
```

❶ Compares balance to amount

❷ The account balance is updated.

❸ The fee is applied and the balance updated again.

Note that there are now two `putfield` instructions, because `safeWithdraw()` takes a `boolean` parameter that determines whether a fee should be charged. The fact that two separate updates occur is what raises the potential for a concurrency bug.

The code for reading the raw balance is very simple:

```
public double getRawBalance();
  Code:
    0: aload_0
    1: getfield      #2  // Field balance:D
    4: dreturn
```

However, this can be interleaved with the withdraw-with-fee code like this:

```
    A14: aload_0
    A15: aload_0
    A16: getfield      #2  // Field balance:D
    A19: iload_1
    A20: i2d
    A21: dsub
    A22: putfield      #2  // Field balance:D   ❶
    A25: iload_2
    A26: ifeq          45
    A29: aload_0
    A30: aload_0
    A31: getfield      #2  // Field balance:D

    // ... Context switch A -> B

     B0: aload_0
     B1: getfield      #2  // Field balance:D   ❷
     B4: dreturn

    // ... Context switch B -> A

    A34: iload_1
    A35: i2d
    A36: aload_0
    A37: getfield      #5  // Field atmFeePercent:D
    A40: dmul
    A41: dsub
    A42: putfield      #2  // Field balance:D
```

❶ The balance written with the amount (but not the fee) deducted

❷ The balance read while the full withdraw is still being processed

With an unsynchronized read, there is the possibility of a *nonrepeatable read*—a value that does not actually correspond to a real state of the system. If you're familiar with

SQL databases, this may remind you of performing a read partway through a database transaction.

NOTE You might be tempted to think, "I know the bytecodes," and optimize your code based on that. You should resist this temptation for several reasons. For example, what happens when you have handed over your code and it is maintained by other developers who do not understand the context or consequences of seemingly harmless code changes?

The conclusion: there is no get-out clause for "just reads." If even one code path fails to use synchronization correctly, the resulting code is not thread-safe and so is incorrect in a multithreaded environment. Let's move on and take a look at how deadlock shows up in bytecode.

5.5 Deadlock revisited

Suppose the bank wants to add the ability to transfer money between accounts into our code. An initial version of this code might look like this:

```
public boolean naiveSafeTransferTo(Account other, int amount) {
   // Check to see amount > 0, throw if not
   synchronized (this) {
      if (balance >= amount) {
         balance = balance - amount;
         synchronized (other) {
            other.rawDeposit(amount);
         }
         return true;
      }
   }
   return false;
}
```

This produces quite a long bytecode listing, so we have shortened it by omitting the by-now-familiar sequence for checking that the balance can support the withdrawal and some synthetic exception-handling blocks.

NOTE There are now two account objects, and each of them has a lock. To be safe, we need to coordinate access to both locks—the lock belonging to `this` and that belonging to `other`.

We will need to deal with *two* pairs of `monitor` instructions, with each pair dealing with the lock of a different object:

```
public boolean naiveSafeTransferTo(Account, int);
  Code:
    0: aload_0
    1: dup
    2: astore_3
    3: monitorenter                    (1)

    // Omit the usual balance checking bytecode

    14: aload_0
    15: aload_0
    16: getfield      #2  // Field balance:D
    19: iload_2
    20: i2d
    21: dsub
    22: putfield      #2  // Field balance:D
    25: aload_1
    26: dup
    27: astore        4
    29: monitorenter                   (2)
    30: aload_1
    31: iload_2
    32: invokevirtual #6  // Method rawDeposit:(I)V
    35: aload         4
    37: monitorexit                    (3)
    38: goto          49

    // Omit exception handling code

    49: iconst_1
    50: aload_3
    51: monitorexit                    (4)
    52: ireturn

    // Omit exception handling code
```

(1) Acquires a lock on this object

(2) Acquires a lock on the other object

(3) Releases the lock on the other object

❹ Releases the lock on this object

Imagine two threads that are trying to transfer money between the same two accounts—let's call the threads `A` and `B`. Let's further suppose that the threads are executing transactions that are labeled by the sending account, so thread `A` is trying to send money from object `A` to object `B` and vice versa:

```
A0: aload_0
A1: dup
A2: astore_3
A3: monitorenter                    ❶

// Omit the usual balance checking bytecode

B0: aload_0
B1: dup
B2: astore_3
B3: monitorenter                    ❷

// Omit the usual balance checking bytecode

B14: aload_0
B15: aload_0
B16: getfield     #2  // Field balance:D
B19: iload_2
B20: i2d
B21: dsub
B22: putfield     #2  // Field balance:D
B25: aload_1
B26: dup
B27: astore       4
B29: ...                            ❸
A14: aload_0
A15: aload_0
A16: getfield     #2  // Field balance:D
A19: iload_2
A20: i2d
A21: dsub
A22: putfield     #2  // Field balance:D
A25: aload_1
A26: dup
A27: astore       4
A29: ...                            ❹
```

1. The lock acquired on account object A (by thread A)

2. The lock acquired on account object B (by thread B)

3. Thread B tries to acquire a lock on object A. It fails and blocks.

4. Thread A tries to acquire a lock on object B. It fails and blocks.

After executing this sequence, neither thread can make progress. Worse yet, only thread A can release the lock on object `A`, and only thread B can release the lock on object `B`, so these two threads are permanently blocked by the synchronization mechanism, and these method calls will never complete. By viewing the deadlock antipattern at a bytecode level, we can see clearly what actually causes it.

5.6 Deadlock resolved, revisited

To solve this problem, as we discussed earlier, we need to ensure that the locks are always acquired in the same order by every thread. One way to do this is by creating an ordering on the threads—say, by introducing a unique account number and implementing the rule: "acquire the lock corresponding to the lowest account ID first."

NOTE For objects that don't have numeric IDs, we will need to do something different, but the general principle of using an unambiguous *total order* still applies.

This method produces a bit more complexity, and to do it completely correctly, we need a guarantee that the account IDs are not reused. We can do this by introducing a `static int` field, which holds the next account ID to be allocated, and updating it only in a `synchronized` method, like this:

```
private static int nextAccountId = 1;

private final int accountId;

private static synchronized int getAndIncrementNextAccountId() {
  int result = nextAccountId;
  nextAccountId = nextAccountId + 1;
  return result;
}

public Account(int openingBalance) {
  balance = openingBalance;
  atmFeePercent = 0.01;
```

```
    accountId = getAndIncrementNextAccountId();
  }

  public int getAccountId() {
    return accountId;
  }
```

We don't need to synchronize the `getAccountId()` method because the field is `final` and can't change, as illustrated here:

```
  public boolean safeTransferTo(final Account other, final int amount) {
    // Check to see amount > 0, throw if not
    if (accountId == other.getAccountId()) {
      // Can't transfer to your own account
      return false;
    }

    if (accountId < other.getAccountId()) {
      synchronized (this) {
        if (balance >= amount) {
          balance = balance - amount;
          synchronized (other) {
            other.rawDeposit(amount);
          }
          return true;
        }
      }
      return false;
    } else {
      synchronized (other) {
        synchronized (this) {
          if (balance >= amount) {
            balance = balance - amount;
            other.rawDeposit(amount);
            return true;
          }
        }
      }
      return false;
    }
  }
```

The resulting Java code is, of course, a little asymmetrical.

NOTE By avoiding holding any locks for any longer than necessary makes it clear which parts of the code actually require locks.

The previous code produces a very long bytecode listing, but let's break it down by parts. First off, we check the ordering of the account IDs:

```
// Elide balance and account equality checks
13: aload_0
14: getfield      #8   // Field accountId:I
17: aload_1
18: invokevirtual #10  // Method getAccountId:()I
21: if_icmpge     91
```

If `A < B` (which it is), then we move on to instruction 24; otherwise, we jump ahead to 91, as follows:

```
24: aload_0
25: dup
26: astore_3
27: monitorenter                    (1)
28: aload_0
29: getfield      #3   // Field balance:D
32: iload_2
33: i2d
34: dcmpl
35: iflt          77                (2)
```

(1) The start of synchronized (this) {

(2) If insufficient funds exist, bails out to offset 77 (further on)

Let's follow the branch where the sending account has sufficient funds to continue, so control falls through to bytecode 38, which is the start of the `balance = balance - amount;` statement in the Java code:

```
38: aload_0
39: aload_0
40: getfield      #3   // Field balance:D
43: iload_2
44: i2d
45: dsub
```

```
46: putfield      #3   // Field balance:D
49: aload_1
50: dup
51: astore        4
53: monitorenter                        ❶
54: aload_1
55: iload_2
56: invokevirtual #9   // Method rawDeposit:(I)V
59: aload         4
61: monitorexit                         ❷
62: goto          73
// Omit exception handling code
73: iconst_1
74: aload_3
75: monitorexit                         ❸
76: ireturn
```

❶ The start of synchronized (other) {

❷ The end of synchronized (other) {

❸ The end of synchronized (this) {

For completeness, let's show the code path used in the case of insufficient balance in the sending account. We basically just unlock the monitor on `this` and return this:

```
77: aload_3
78: monitorexit                         ❶
79: goto          89
// Omit exception handling code
89: iconst_0
90: ireturn
```

❶ The end of synchronized (this) {

Note that some of the instructions (such as `invoke` and `monitor` instructions) may throw exceptions, so we are, as usual, ignoring the bytecode handlers for those exceptions. The rest of the method looks like this:

```
91: aload_1
// ...
// Highly similar, but for the other branch
```

Let's look at what happens with two threads, remembering that the account ID of `A < B`.

We now have one additional complication: the local variables (used in instructions such as `aload_0`) are different between the two threads. To draw out this distinction, we'll slightly mangle the bytecode by labeling the local variable with the thread as well, so we'll write `aload_A0` and `aload_A1` for clarity:

```
    A24: aload_A0
    A25: dup
    A26: astore_A3
    A27: monitorenter                          (1)

    // Elide balance check

    A38: aload_A0
    A39: aload_A0
    A40: getfield     #3   // Field balance:D

// ....         Context switch A -> B

    B91: aload_B1
    B92: dup
    B93: astore_B3
    B94: monitorenter                          (2)

// ....         Context switch B -> A

    A43: iload_A2
    A44: i2d
    A45: dsub
    A46: putfield     #3   // Field balance:D
    A49: aload_A1
    A50: dup
    A51: astore       A4
    A53: monitorenter                          (3)
    A54: aload_A1
    A55: iload_A2
    A56: invokevirtual #9   // Method rawDeposit:(I)V
    A59: aload        A4
    A61: monitorexit                           (4)
    A62: goto         73

    // Omit exception handling code

    A73: iconst_A1
```

```
        A74: aload_A3
        A75: monitorexit                    (5)

  // ....          Context switch A -> B

        B95: aload_B0
        B96: dup
        B97: astore       B4
        B99: monitorenter
        // ...
       B132: ireturn

  // ....          Context switch B -> A

        A76: ireturn
```

(1) The lock acquired on object A by thread A

(2) The lock attempted on object A by thread B: blocks

(3) The lock acquired on object B by thread A

(4) The lock released on object B by thread A

(5) The lock released on object A by thread A: thread B can resume

This is, without doubt, a complex listing. The key insight is that `A0 == B1`, so locking these two objects will always induce a blocking call in the second thread. The invariant `A < B` ensures that thread B is sent down the alternate branch.

5.7 Volatile access

What does `volatile` look like in bytecode? Let's take a look at an important pattern—*Volatile Shutdown*—to help answer this.

The Volatile Shutdown pattern helps solve the problem of interthread communication that we touched upon earlier when we met the dangerous and deprecated `stop()` method. Consider a simple class that is responsible for doing some work. In the simplest case, we will assume that work comes in discrete units, with a well-defined "complete" status for each unit, as shown next:

```
public class TaskManager implements Runnable {
   private volatile boolean shutdown = false;

   public void shutdown() {
      shutdown = true;
   }

   @Override
   public void run() {
      while (!shutdown) {
         // do some work - e.g. process a work unit
      }
   }
}
```

The intent of the pattern is hopefully clear. All the time the `shutdown` flag is `false`, work units will continue to be processed. If it ever flips to `true`, then the `TaskManager` will, after it has completed its current work unit, exit the `while` loop and the thread will exit cleanly, in a "graceful shutdown."

The more subtle point is derived from the Java Memory Model: any write to a volatile variable Happens-Before all subsequent reads of that variable. As soon as another thread calls `shutdown()` on the `TaskManager` object, the flag is changed to `true` and the effect of that change is guaranteed to be visible on the next read of the flag—before the next work unit is accepted.

The Volatile Shutdown pattern produces bytecode like this:

```
public class TaskManager implements java.lang.Runnable {
  private volatile boolean shutdown;

  public TaskManager();
    Code:
      0: aload_0
      1: invokespecial #1         // Method java/lang/Object."<init>":()V
      4: aload_0
      5: iconst_0
      6: putfield      #2         // Field shutdown:Z
      9: return

  public void shutdown();
    Code:
      0: aload_0
```

```
       1: iconst_1
       2: putfield      #2         // Field shutdown:Z
       5: return

  public void run();
    Code:
       0: aload_0
       1: getfield      #2         // Field shutdown:Z
       4: ifne          10
       7: goto          0
      10: return
}
```

If you look carefully, you can see that the volatile nature of `shutdown` does not appear anywhere except in the field definition. There are no additional clues on the opcodes—and it is accessed using the standard `getfield` and `putfield` opcodes.

NOTE `volatile` is a hardware access mode and produces a CPU instruction that says to ignore the cache hardware and instead read or write directly from main memory.

The only difference is in how `putfield` and `getfield` behave—the implementation of the bytecode interpreter will have separate code paths for volatile and standard fields.

In fact, *any* piece of physical memory can be accessed in a volatile manner, and—as we will see later on—this is not the only access mode possible. The volatile case is merely a common case of access semantics that James Gosling and the original designers of Java chose to encode in the core of the language, by making it a keyword that can apply to fields.

Concurrency is one of the most important features of the Java platform, and a good developer will increasingly need a solid understanding of it. We've reviewed the underpinnings of Java's concurrency and the design forces that occur in multithreaded systems. We've discussed the Java Memory Model and low-level details of how the platform implements concurrency.

This chapter isn't intended to be a complete statement of everything you'll ever need to know about concurrency—it's enough to get you started and give you an appreciation of what you'll need to learn more about, and stop you from being dangerous when writing concurrent code. But you'll need to know more than we can cover here if you're going to be a truly first-rate developer of multithreaded code. A number of excellent books about

nothing but Java concurrency are out there. One of the best is *Java Concurrency in Practice* by Brian Goetz and others (Addison-Wesley Professional, 2006).

Summary

- Java's threads are a low-level abstraction.
- Multithreading is present even in the Java bytecode.
- The Java Memory Model is very flexible but makes minimal guarantees.
- Synchronization is a cooperative mechanism—all threads must participate to achieve safety.
- Never use `Thread.stop()` or `Thread.suspend()`.

Previous chapter
< 4 Class files and bytecode

Next chapter
6 JDK concurrency libraries >